

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

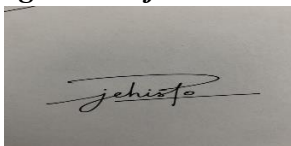
Annexure A

Scenario Based Assignment

Code of Conduct:

Jehisto rijin J / 192411022 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature appears to be 'Jehisto' written in a cursive, flowing style.

Annexure A

Scenario Based Assignment

1. **Scenario** : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario**: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. ANSWER

1. Greedy Best-First Search (GBFS)

- GBFS uses only the heuristic $h(n)$ (straight-line distance) to select the next path.
- It quickly moves toward the destination without considering actual travel cost.
- Advantages: Fast, simple, and suitable for urgent decisions.
- Disadvantages: Ignores terrain, weather, and blocked roads, so it may choose unsafe or non-optimal routes.

2. A* Search

- A* uses $f(n) = g(n) + h(n)$, where:
 - $g(n)$ = Actual travel cost.
 - $h(n)$ = Estimated distance to the goal.
- It balances both actual cost and heuristic information.
- It finds the shortest and safest path while adapting to changing conditions.

3. Impact of Heuristic Accuracy

- GBFS: An accurate heuristic gives faster results, while an inaccurate heuristic may lead to poor route selection.
- A*: An accurate heuristic improves speed, and even with a less accurate heuristic, A* can still find an optimal path (with an admissible heuristic).

4. Effect of Dynamic Changes

- Floods may block roads or increase travel costs.
- GBFS: Performs poorly because it may repeatedly choose blocked paths.
- A*: Updates path costs and replans efficiently when new information is available.

5. Recommendation

A* is the most suitable algorithm because it:

- Considers both travel cost and estimated distance.
- Finds an optimal and safe route.
- Adapts better to dynamic environments.
- Provides reliable navigation for emergency medicine delivery despite slightly higher computation time.

Conclusion: While GBFS is faster, A* is preferred for flood-affected regions because it offers better optimality, safety, and adaptability, making it ideal for real-time emergency drone navigation.

2. ANSWER

1. Formulation as an Optimization Problem

The objective is to optimize traffic signal timings to:

- Minimize total waiting time.
- Reduce fuel consumption.
- Decrease traffic congestion.

Traditional search methods are inefficient because:

- The search space is very large.
- Traffic conditions change continuously.
- Exploring every possible signal combination takes too much time.

2. Hill Climbing

- Hill Climbing starts with a signal timing plan.
- It makes small changes and selects the solution with better performance.
- This process continues until no better neighboring solution is found.

Limitations:

- May get stuck in a local optimum.
- Cannot guarantee the best global solution.
- Performs poorly in dynamic traffic conditions.

3. Local Maxima, Plateaus, and Ridges

- Local Maxima: The system finds a good solution but not the best one (some intersections improve while overall traffic remains congested).
- Plateaus: Many signal timings produce the same result, so the algorithm cannot decide where to move.
- Ridges: The best solution requires several coordinated changes, but Hill Climbing changes only one step at a time.

4. Simulated Annealing / Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima and explores better solutions.
- Suitable for dynamic environments.

Genetic Algorithm

- Uses a population of solutions.
- Applies selection, crossover, and mutation to generate better signal timings.
- Finds high-quality solutions for large and complex traffic networks.

5. Recommendation

Genetic Algorithm is the best choice because it:

- Handles very large search spaces.
- Avoids local optima.
- Adapts well to changing traffic patterns.
- Produces efficient and scalable traffic signal optimization.

Conclusion: While Hill Climbing is simple and fast, Genetic Algorithms provide better scalability, adaptability, and overall traffic optimization for smart city traffic management.

3. ANSWER

1. Why Online Search is Required

- The rover has no complete map of Mars.
- It discovers obstacles and paths while moving.
- Decisions must be made in real time using current information.

- Hence, online search is more suitable than offline search.

2. Challenges of Partial Observability and Dynamic Environment

- The rover has limited sensors and cannot see the entire terrain.
- Hazards such as rocks and craters are detected only when nearby.
- Unknown terrain increases uncertainty.
- The rover must continuously adapt to new information.

3. Building and Updating the Internal Model

- The rover starts with little or no knowledge.
- It collects data using cameras and sensors.
- As it moves, it stores information about safe paths and obstacles.
- The internal map is updated continuously to improve future decisions.

4. Online Search vs. Uninformed and Informed Search

Online Search	Uninformed Search	Informed Search
Learns while exploring	Requires a complete map	Uses heuristic knowledge
Works in unknown environments	Suitable only for known environments	Better than uninformed but still needs map information
Adapts to changes	Less adaptable	Limited adaptability

5. Recommendation

Online Search Agent is the best approach because it:

- Works in unknown and partially observable environments.
- Updates knowledge continuously.
- Adapts to newly discovered hazards.
- Ensures safe and reliable navigation while collecting samples.

Conclusion: Online Search is the most suitable method for a Mars rover because it provides real-time decision-making, adaptability, and safety in uncertain environments.

4. ANSWER

1. Formulation as a Constraint Satisfaction Problem (CSP)

The exam timetable is modeled as a Constraint Satisfaction Problem (CSP) where the goal is to assign exam slots to courses while satisfying all constraints.

2. Variables, Domains, and Constraints

Variables

- Courses/Subjects to be scheduled.

Domains

- Available exam time slots and exam halls.

Constraints

- No student should have overlapping exams.
- Limited number of exam halls.
- Some subjects must be scheduled before others.
- Faculty must be available during the exam.
- Special accommodations must be provided for eligible students.

3. Backtracking Search

- Assigns a time slot and hall to one course at a time.
- Checks whether all constraints are satisfied.
- If a conflict occurs, it goes back (backtracks) and tries another assignment.
- Continues until a valid timetable is found.

4. Constraint Propagation (Forward Checking)

- After assigning a course, forward checking removes invalid time slots from the remaining courses.
- Detects conflicts early.

- Reduces unnecessary backtracking.
- Improves the speed of timetable generation.

5. Real-World Challenges and Best Strategy

Challenges

- Large number of courses and students.
- Multiple constraints increase complexity.
- Last-minute changes in faculty or exam halls.
- Special accommodation requirements.

Recommendation

Backtracking with Forward Checking (Constraint Propagation) is the best strategy because it:

- Finds valid timetables efficiently.
- Reduces search time by eliminating invalid choices early.
- Scales better for large universities with many constraints.

Conclusion: Using CSP with Backtracking and Forward Checking provides an efficient, scalable, and reliable solution for automatic exam timetable generation.

5. ANSWER

1. Minimax Algorithm

- Minimax models the game as a two-player decision problem.
- The AI (MAX) tries to maximize its score, while the opponent (MIN) tries to minimize it.
- The AI evaluates possible moves and selects the best one assuming the opponent plays optimally.

2. Computational Challenges

- The game tree has a very large number of possible moves.

- Searching every branch requires high time and memory.
- Real-time games have strict time limits, making full search impractical.

3. Alpha-Beta Pruning

- Alpha-Beta pruning is an optimization of Minimax.
- It removes branches that cannot affect the final decision.
- Alpha (α): Best value found for the MAX player.
- Beta (β): Best value found for the MIN player.
- If $\alpha \geq \beta$, the remaining branches are pruned.
- This reduces the number of nodes explored, making the search much faster while still producing the same optimal result.

4. Evaluation Function

The evaluation function scores each game state based on:

- Number of units available.
- Health of units.
- Resources collected.
- Control of important locations.
- Distance to enemy base.
- Overall attack and defense strength.

These factors help the AI estimate the quality of a move when the game cannot be searched to the end.

5. Recommendation

Use Minimax with Alpha-Beta Pruning and an Evaluation Function because it:

- Produces optimal decisions.
- Reduces unnecessary computations.
- Meets real-time decision requirements.
- Balances accuracy and speed for complex strategy games.

Conclusion: Minimax with Alpha-Beta Pruning is the best approach as it maintains optimal gameplay while significantly improving performance for large game trees.